Extended Kalman Filter for a 4-TurtleBot3 Swarm with Overhead ArUco Camera Feedback

Mathematical Theory and Implementation Guide

ROS 2 Jazzy / Gazebo Harmonic

June 29, 2026

Contents

1 Motivation and Problem Statement 2

2 System Model 2
  2.1 Unicycle Kinematics (Process Model) . . . 2
  2.2 Measurement Model (Overhead ArUco Camera) . . . 3

3 Extended Kalman Filter — Step-by-Step Derivation 3
  3.1 Step 1: Prediction (Runs Every Control Cycle) . . . 3
  3.2 Step 2: Measurement Update (Only When Marker Detected) . . . 3

4 Convergence Detector (Problem 3) 4

5 Integration with the Double-Angle CBF 5

6 Data Flow and Architecture 5

7 Noise Parameter Tuning 6

8 How to Run 6

9 Summary of Fixes 7

1 Motivation and Problem Statement

The formation controller `multi_unicycle_controller_double_angle_experiment.py` drives four TurtleBot3 Burger robots into a desired geometric formation using the *Double-Angle Visual-Cone Control Barrier Function* (CBF) algorithm. Pose feedback comes from an overhead RealSense camera that tracks ArUco markers mounted on each robot via the `aruco_opencv` ROS 2 package, which publishes TF frames:

$$\texttt{ground\_origin} \longrightarrow \texttt{marker\_}i, \quad i \in \{0, 1, 2, 3\}.$$

Three operational problems degrade performance:

1. **Marker detection loss.** When a robot moves near the camera's edge, is occluded by another robot, or the lighting changes, the ArUco detector fails for several frames. The controller then has *no state estimate* and either freezes or uses a stale measurement, causing robots to drift unpredictably.
2. **Trajectory spikes.** Perspective distortion, motion blur, and marker orientation ambiguity occasionally cause the `solvePnP` solver to produce a spurious pose estimate that is tens of centimetres away from the true position. Because the state is set directly from each raw TF measurement, these outliers propagate immediately into the control law, causing velocity spikes and transient CBF violations.
3. **Robots do not stop after convergence.** The original code applies the CBF law continuously. Once the formation error is small the CBF gradient ξ_{ij} becomes near-zero but non-zero, producing tiny, persistent velocity commands that keep the robots oscillating around their formation points indefinitely.

The **Extended Kalman Filter** (EKF) addresses problems 1 and 2. A *convergence detector* addresses problem 3.

2 System Model

2.1 Unicycle Kinematics (Process Model)

Each TurtleBot3 is modelled as a *unicycle*:

$$\dot{\boldsymbol{x}}_i = \begin{bmatrix} \dot{x}_i \\ \dot{y}_i \\ \dot{\theta}_i \end{bmatrix} = \begin{bmatrix} v_i \cos\theta_i \\ v_i \sin\theta_i \\ \omega_i \end{bmatrix}, \qquad \boldsymbol{x}_i = \begin{bmatrix} x_i \\ y_i \\ \theta_i \end{bmatrix} \in \mathbb{R}^3, \tag{1}$$

where $v_i \in \mathbb{R}$ is the commanded linear speed and $\omega_i \in \mathbb{R}$ is the commanded angular speed (both published on `/bot_i/cmd_vel`).

Discretising with time step Δt (Euler forward):

$$\boldsymbol{x}_i^{k+1} = \boldsymbol{f}\left(\boldsymbol{x}_i^k, \boldsymbol{u}_i^k\right) = \begin{bmatrix} x_i^k + v_i^k \cos\theta_i^k \,\Delta t \\ y_i^k + v_i^k \sin\theta_i^k \,\Delta t \\ \theta_i^k + \omega_i^k \,\Delta t \end{bmatrix}. \tag{2}$$

The process noise models wheel-slip, actuator lag, and timing jitter:

$$\boldsymbol{w}_i^k \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}), \qquad \boldsymbol{Q} = \mathrm{diag}(q_{xy}, q_{xy}, q_\theta).$$

2.2 Measurement Model (Overhead ArUco Camera)

The ceiling camera observes each robot's marker and, after TF lookup, provides:

$$\boldsymbol{z}_i^k = \boldsymbol{h}\big(\boldsymbol{x}_i^k\big) + \boldsymbol{v}_i^k = \boldsymbol{x}_i^k + \boldsymbol{v}_i^k, \tag{3}$$

where $\boldsymbol{v}_i^k \sim \mathcal{N}(\mathbf{0}, \boldsymbol{R})$ represents camera noise:

$$\boldsymbol{R} = \operatorname{diag}(r_{xy},\, r_{xy},\, r_\theta).$$

Because $\boldsymbol{h}$ is *linear* (identity), the measurement Jacobian is:

$$\boldsymbol{H}_i = \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}_i} = \boldsymbol{I}_3.$$

3 Extended Kalman Filter — Step-by-Step Derivation

An independent EKF is run for *each robot* at 50 Hz. Let $\hat{\boldsymbol{x}}_i^{k|k}$ denote the posterior state estimate and $\boldsymbol{P}_i^{k|k}$ the posterior covariance.

3.1 Step 1: Prediction (Runs Every Control Cycle)

Even when the ArUco marker is **not visible**, the EKF predicts the robot's state from its kinematic model using the *last commanded inputs* $\boldsymbol{u}_i^k = (v_i^k,\, \omega_i^k)$.

State prediction:

$$\hat{\boldsymbol{x}}_i^{k+1|k} = \boldsymbol{f}\left(\hat{\boldsymbol{x}}_i^{k|k},\, \boldsymbol{u}_i^k\right). \tag{4}$$

Jacobian of the process model:

$$\boldsymbol{F}_i^k = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}_i}\right|_{\hat{\boldsymbol{x}}_i^{k|k}} = \begin{bmatrix} 1 & 0 & -v_i^k \sin\hat{\theta}_i^{k|k}\,\Delta t \\ 0 & 1 & v_i^k \cos\hat{\theta}_i^{k|k}\,\Delta t \\ 0 & 0 & 1 \end{bmatrix}. \tag{5}$$

Covariance prediction:

$$\boldsymbol{P}_i^{k+1|k} = \boldsymbol{F}_i^k\, \boldsymbol{P}_i^{k|k}\, \left(\boldsymbol{F}_i^k\right)^\top + \boldsymbol{Q}. \tag{6}$$

Why this fixes Problem 1: When the marker is lost, only equations (2)–(6) execute. The estimated state keeps evolving based on the robot's actual commands, so the control law continues to receive a physically consistent pose estimate rather than a frozen or invalid one. The growing covariance $\boldsymbol{P}$ also signals increasing uncertainty, which naturally reduces the Kalman gain when the marker reappears.

3.2 Step 2: Measurement Update (Only When Marker Detected)

When a valid ArUco TF measurement $\boldsymbol{z}_i^k$ arrives:

Innovation (measurement residual):

$$\tilde{\boldsymbol{y}}_i^k = \boldsymbol{z}_i^k - \boldsymbol{H}_i\, \hat{\boldsymbol{x}}_i^{k|k-1} = \boldsymbol{z}_i^k - \hat{\boldsymbol{x}}_i^{k|k-1}. \tag{7}$$

Angles are wrapped to $(-\pi, \pi]$ to avoid discontinuities:

$$\tilde{y}_{\theta,i}^k = \operatorname{atan2}\big(\sin \tilde{y}_{\theta,i}^k,\ \cos \tilde{y}_{\theta,i}^k\big)\,.$$

Innovation covariance:

$$\boldsymbol{S}_i^k = \boldsymbol{H}_i\, \boldsymbol{P}_i^{k|k-1}\, \boldsymbol{H}_i^\top + \boldsymbol{R} = \boldsymbol{P}_i^{k|k-1} + \boldsymbol{R}. \tag{8}$$

Mahalanobis outlier gate (fixes Problem 2): A measurement is accepted **only if**

$$\big(\tilde{\boldsymbol{y}}_i^k\big)^\top \big(\boldsymbol{S}_i^k\big)^{-1} \tilde{\boldsymbol{y}}_i^k \ \leq\ \gamma, \tag{9}$$

where $\gamma = 11.34$ is the 99% quantile of the χ_3^2 distribution (3 degrees of freedom). A spike corresponds to a measurement that lies many standard deviations from the predicted state; it yields a large Mahalanobis distance and is rejected, preventing it from corrupting the estimate.

Kalman gain:

$$\boldsymbol{K}_i^k = \boldsymbol{P}_i^{k|k-1}\, \big(\boldsymbol{S}_i^k\big)^{-1}. \tag{10}$$

State update:

$$\hat{\boldsymbol{x}}_i^{k|k} = \hat{\boldsymbol{x}}_i^{k|k-1} + \boldsymbol{K}_i^k\, \tilde{\boldsymbol{y}}_i^k. \tag{11}$$

Covariance update (Joseph form, numerically stable):

$$\boldsymbol{P}_i^{k|k} = \big(\boldsymbol{I} - \boldsymbol{K}_i^k \boldsymbol{H}_i\big)\, \boldsymbol{P}_i^{k|k-1}\, \big(\boldsymbol{I} - \boldsymbol{K}_i^k \boldsymbol{H}_i\big)^\top + \boldsymbol{K}_i^k \boldsymbol{R}\, \big(\boldsymbol{K}_i^k\big)^\top. \tag{12}$$

4 Convergence Detector (Problem 3)

The formation position error for each edge $(i, j) \in \mathcal{N}$ is:

$$\boldsymbol{e}_{ij} = \boldsymbol{p}_i - \boldsymbol{p}_j - \boldsymbol{e}_{ij}^\star,$$

where $\boldsymbol{e}_{ij}^\star$ is the desired relative displacement from $\boldsymbol{p}^\star$.

The mean formation error is:

$$\varepsilon_{\text{form}} = \frac{1}{|\mathcal{N}|} \sum_{(i,j)\in\mathcal{N}} \|\boldsymbol{e}_{ij}\|. \tag{13}$$

The controller declares **convergence** when:

$$\varepsilon_{\text{form}} < \varepsilon_{\text{tol}} \quad \text{for } N_{\text{conv}} \text{ consecutive cycles,}$$

after which all robot velocities are set to zero and the control loop halts. Using $N_{\text{conv}} = 50$ (at 50 Hz, this is 1 second of stable formation) prevents premature stopping due to momentary noise crossings.

5 Integration with the Double-Angle CBF

The EKF-smoothed state $\hat{\boldsymbol{x}}_i = (\hat{x}_i, \hat{y}_i, \hat{\theta}_i)$ replaces the raw TF values in the formation controller:

Step 1: Assemble the stacked position vector:

$$\boldsymbol{p} = \begin{bmatrix} \hat{x}_1 & \hat{y}_1 & \hat{x}_2 & \hat{y}_2 & \hat{x}_3 & \hat{y}_3 & \hat{x}_4 & \hat{y}_4 \end{bmatrix}^\top \in \mathbb{R}^8.$$

Step 2: Compute edge vectors via the incidence matrix $\bar{\boldsymbol{H}}_{\mathcal{N}}$:

$$\boldsymbol{e}_{\mathcal{N}} = \bar{\boldsymbol{H}}_{\mathcal{N}} \, \boldsymbol{p}.$$

Step 3: For each edge ℓ compute the double-angle CBF gradient:

$$\boldsymbol{R}_\psi^{(\ell)}, \quad \xi_\ell,$$

using the projection geometry of the visual cone.

Step 4: Compute the Cartesian velocity commands:

$$\dot{\boldsymbol{p}} = -\bar{\boldsymbol{H}}_{\mathcal{N}}^\top \, \boldsymbol{R}_\psi \, \boldsymbol{\xi}.$$

Step 5: Convert to unicycle inputs using the look-ahead transformation (parameter $\ell = 0.1$ m):

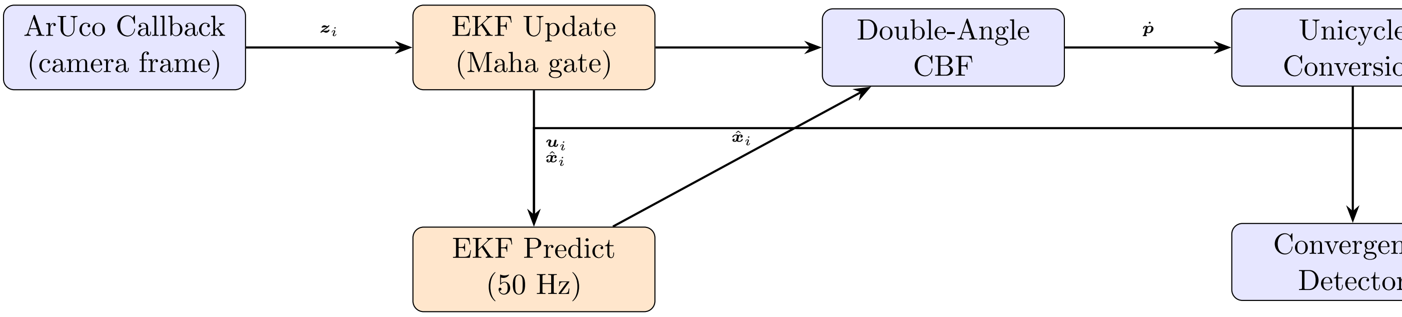
$$v_i = 2\Big(\dot{p}_{x,i} \cos\hat{\theta}_i + \dot{p}_{y,i} \sin\hat{\theta}_i\Big), \tag{14}$$

$$\omega_i = \frac{-\dot{p}_{x,i} \sin\hat{\theta}_i + \dot{p}_{y,i} \cos\hat{\theta}_i}{\ell}. \tag{15}$$

Step 6: Feed (v_i, ω_i) back to the EKF as $\boldsymbol{u}_i$ for the *next prediction step*.

Step 7: Publish (v_i, ω_i) as `TwistStamped` on `/bot_i/cmd_vel`.

6 Data Flow and Architecture


ArUco Callback
(camera frame)
$\boldsymbol{z}_i$
EKF Update
(Maha gate)
Double-Angle
CBF
$\dot{\boldsymbol{p}}$
Unicycle
Conversio
$\boldsymbol{u}_i$
$\hat{\boldsymbol{x}}_i$
$\hat{\boldsymbol{x}}_i$
EKF Predict
(50 Hz)
Convergen
Detector

7 Noise Parameter Tuning

Parameter	Symbol	Default	Meaning
q_{xy}	`Q_XY`	0.001	Process noise variance (position)
q_θ	`Q_THETA`	0.002	Process noise variance (heading)
r_{xy}	`R_XY`	0.004	Measurement noise variance (position)
r_θ	`R_THETA`	0.008	Measurement noise variance (heading)
γ	`GATE_THRESHOLD`	11.34	Mahalanobis gate (χ^2_3, 99%)
ε_{tol}	`FORMATION_TOL`	0.03 m	Convergence threshold
N_{conv}	`CONVERGE_CYCLES`	50	Stability window (cycles)

Tuning Guidelines

- **Estimate too sluggish (slow to react to fast motion):** Increase q_{xy}, q_θ — trust the kinematic model less.
- **Measurements very noisy (camera vibration, bad lighting):** Increase r_{xy}, r_θ — trust measurements less.
- **Valid measurements rejected as outliers:** Increase γ (e.g. to 16.27 for 99.9%).
- **Robots stop too early:** Reduce ε_{tol} or increase N_{conv}.
- **Robots never stop:** Increase ε_{tol} slightly (e.g. 0.05 m).

8 How to Run

1. Source ROS 2 and the workspace:

```
source /opt/ros/jazzy/setup.bash
source ~/Downloads/ros2_ws/turtlebot/install/setup.bash
export ROS_DOMAIN_ID=30   # real robots
```

2. Launch the ArUco tracker (overhead camera):

```
ros2 launch /home/gaurav/aruco_config/tracker.launch.xml
```

3. Run the EKF controller:

```
python3 ~/Downloads/ros2_ws/experiment/multi_unicycle_ekf.py
```

4. The node prints EKF state, Mahalanobis distance, and formation error every 0.5 seconds. When the formation converges, all robots stop automatically.

For **Gazebo simulation**, change `ROS_DOMAIN_ID=0` and replace the ArUco measurement source with `/bot_N/ground_truth_odom`.

9 Summary of Fixes

Problem	Root Cause	EKF Fix
ArUco marker lost	Raw TF used directly; state frozen when lost	EKF *predict-only* propagates state using commanded (v_i, ω_i); covariance grows to reflect uncertainty
Trajectory spikes	Outlier TF poses fed directly to CBF	Mahalanobis gate (9) rejects measurements more than 3σ from prediction
Robots do not stop	CBF gradient never reaches exact zero	Convergence detector counts cycles with $\varepsilon_{\text{form}} < \varepsilon_{\text{tol}}$; calls `stop_all()` after N_{conv} cycles